

DSA Exam-Style Q&A; Cheatsheet

1. What is an Abstract Data Type (ADT)?

An ADT defines the logical model of a data structure and the operations performed on it, without specifying implementation details.

2. How is a User Defined Type (UDT) different from an ADT?

A UDT is a custom data type defined by the programmer (e.g., structs or classes), while an ADT describes behavior through operations.

3. Why is time complexity important in algorithm design?

It helps estimate performance and scalability before implementation, comparing algorithms efficiently.

4. What is the worst-case time complexity of linear search and why?

$O(n)$, because in the worst case the element is at the last position or absent, requiring all elements to be checked.

5. Why does binary search require a sorted array?

Because it repeatedly divides the search space in half, which only works if the data is in order.

6. What is the time complexity of binary search and why?

$O(\log n)$, as the array is halved with each comparison.

7. What is the difference between static and dynamic arrays?

Static arrays have fixed size at compile time; dynamic arrays resize at runtime using reallocation.

8. What is amortized complexity in dynamic arrays?

Amortized $O(1)$ insertion; resizing is costly but happens infrequently.

9. Why is linked list insertion at the head $O(1)$?

Only pointer adjustments are required without shifting elements.

10. Why is random access not possible in linked lists?

Nodes aren't stored contiguously; traversal is necessary to reach a position.

11. What does LIFO stand for in stacks?

Last In, First Out — the last inserted element is removed first.

12. Why is `pop()` in stacks considered $O(1)$?

Removing the top element requires constant time.

13. What does FIFO stand for in queues?

First In, First Out — elements are removed in the order they were inserted.

14. What is the purpose of enqueue and dequeue operations?

Enqueue inserts at the rear, dequeue removes from the front.

15. What is hashing and why is it used?

Hashing maps keys to indices using a hash function for fast search, insert, and delete.

16. What is a collision in hashing?

When two keys map to the same index in the hash table.

17. How does chaining handle collisions?

By storing multiple elements in a linked list at the same hash index.

18. What is the average time complexity of hash table search?

$O(1)$ average case, assuming good hash function and low load factor.

19. What is Big-O notation used for?

To represent the upper bound of time complexity in worst-case scenarios.

20. What is the time complexity of inserting at the end of an array?

$O(1)$ if space is available; $O(n)$ if resizing occurs.